```
          context.log.info(s"not handling ${event.toString}")
          Behaviors.same

      }
      .receiveSignal {
        case (context, PostStop) =>
          Cluster(context.system).subscriptions ! Unsubscribe(
            context.self)                                          ❸
          Behaviors.stopped
      }
  }
}
```

❶ Subscribes to the cluster domain events on actor creation

❷ Listens for cluster domain events

❸ Unsubscribes after receiving the stopped signal

Together with `Cluster.manager`, which is used to shut down or exit nodes in the cluster, `Cluster.subscriptions` completes the available API of `akka.cluster.typed.Cluster`. While the former is used almost exclusively in Akka internals, the latter is used in applications.

With this understanding of the basics of an Akka cluster, let's dive into the details of a simple application and distinguish what belongs to the cluster functionality and what belongs to the application domain.

## *8.4 Clustered job processing*

The following application uses two main types of actors: the master and the worker, each living in different nodes depending on whether the node has the role of director or aggregator. The master produces texts that need to be grouped and counted by words. To accomplish this, the master delegates the grouping and counting of each text to the workers while aggregating only the result of each worker's work.